SOFTWARE ENGINEERING PROJECT REPORT

UNIVERSAL LOG PROCESSING FRAMEWORK WITH AI ANALYSIS

Design, Implementation and Testing

Submitted in partial fulfilment of the requirements for the award of
BSc (Hons) Computer Science

Submitted by
Khushveer Kaur, Gaurav, Mayank Nautiyal and Anjali

24020570052, 24020570058, 24020570045, 24020570026

Under the guidance of
Dr. Kamlesh Kumar Raghuvanshi

Department of Computer Science
Ramanujan College, University of Delhi

Kalkaji, New Delhi, Delhi 110019

ACADEMIC SESSION 2026–2027

CERTIFICATE

This is to certify that the project report entitled “Universal Log Processing Framework with AI Analysis” is a bona fide record of the work carried out by Khushveer Kaur, Gaurav, Mayank Nautiyal and Anjali (University Roll Nos. 24020570052, 24020570058, 24020570045 and 24020570026) under my supervision and guidance, in partial fulfilment of the requirements for the award of the B.Sc. (Hons.) Computer Science degree during the academic session 2026–2027.

To the best of my knowledge, the work presented in this report has been completed by the students and has not been submitted, in whole or in part, for the award of any other degree or diploma.

Project Guide: _____ Head of Department: _____

Date: _____ Place: _____

DECLARATION

We hereby declare that the project work presented in this report, entitled “Universal Log Processing Framework with AI Analysis”, was carried out by our group under the guidance of Dr. Kamlesh Kumar Raghuvanshi. The implementation, observations and explanations in the report are based on the project source code, application behaviour and tests conducted during development.

We further declare that the external tools, libraries, platforms and reference materials used in the project have been acknowledged. Any implementation limitations or results that could not be independently verified have been identified in the report.

Signatures of Students: _____

Names: Khushveer Kaur, Gaurav, Mayank Nautiyal and Anjali

Date: _____

ACKNOWLEDGEMENT

We would like to thank Dr. Kamlesh Kumar Raghuvanshi for his guidance and feedback throughout this project. His suggestions helped us define the project scope, make design decisions and improve the implementation.

We are also thankful to the faculty members of the Department of Computer Science, Ramanujan College, for providing the academic environment and resources needed for this work. We acknowledge the developers and maintainers of the open-source technologies used in the project, including Python, Apache Kafka, Elasticsearch and Streamlit, along with the providers of the machine-learning and language-model services.

Finally, we thank our families and friends for their encouragement and support during the project.

ABSTRACT

Modern software applications generate operational records from multiple services, components and infrastructure layers. These records are useful for troubleshooting, security review and reliability engineering, but they are often heterogeneous, high-volume and difficult to investigate manually. Differences in format and field naming make correlation difficult, while large volumes of repetitive messages can delay the identification of meaningful failures.

The Universal Log Processing Framework with AI Analysis is designed as an integrated pipeline for collecting, parsing, validating, normalising, classifying, storing and investigating application logs. The implemented collector reads log files and maintains checkpoint information so that collection can resume from previously recorded positions. The parser supports JSON, XML, CSV, Syslog, CEF and LEEF input paths, constructs a canonical log representation, validates required fields and masks sensitive values before downstream processing. Apache Kafka provides asynchronous message transport between processing stages. The processing service assigns a classification and priority to accepted records and routes invalid events to a dead-letter topic.

Processed records are indexed in Elasticsearch for structured search and dashboard analytics. A semantic retrieval layer uses Sentence Transformers to create 384-dimensional embeddings with the all-MiniLM-L6-v2 model and stores vectors in Elasticsearch. The retrieval-augmented generation pipeline obtains similar historical log records and supplies the evidence to a Groq-hosted language model for diagnostic assistance. A Streamlit application exposes six functional areas: overview dashboard, log explorer, AI assistant, AI investigations, incident centre and dead-letter queue.

The project demonstrates an end-to-end architecture in which deterministic log processing and search are combined with evidence-supported AI assistance. The application is designed to support investigation; generated diagnoses are advisory and do not independently confirm that an incident has been resolved.

Keywords: log processing, Apache Kafka, Elasticsearch, log normalisation, data masking, vector search, embeddings, retrieval-augmented generation, AI diagnosis, Streamlit.

TABLE OF CONTENTS

Contents

CERTIFICATE 2
DECLARATION 3
ACKNOWLEDGEMENT 4
ABSTRACT 5
TABLE OF CONTENTS 6
LIST OF TABLES AND FIGURES 9
1. INTRODUCTION 10
1.1 Background 10
1.2 Problem Statement 10
1.3 Project Objectives 10
1.4 Scope 10
1.5 Intended Users 11
1.6 Organisation of the Report 11
2. PROJECT PLANNING AND FEASIBILITY 11
2.1 Development Approach 11
2.2 Technical Feasibility 11
2.3 Operational Feasibility 11
2.4 Economic Feasibility 11
2.5 Schedule and Deliverables 11
2.6 Risk Assessment 12
3. SOFTWARE REQUIREMENTS SPECIFICATION 12
3.1 Purpose 12
3.2 Product Perspective 12
3.3 Functional Requirements 12
3.4 Non-functional Requirements 12
3.5 External Interface Requirements 13
3.6 Hardware and Software Environment 13
3.7 Assumptions and Constraints 13
4. SYSTEM ANALYSIS AND DESIGN 13
4.1 Architectural Overview 13
4.2 Context-Level Data Flow 14
4.3 Level-1 Data Flow 14
4.4 Canonical Record Design 15

4.5 Processing Workflow 15
4.6 Use-Case Model 16
4.7 Logical Data Model 16
4.8 Major Components 17
5. LOG COLLECTION AND NORMALISATION 17
5.1 File Discovery and Collection 17
5.2 Supported Input Formats 18
5.3 Parsing and Canonicalisation 18
5.4 Validation 18
5.5 Sensitive-Data Masking 18
5.6 Canonical Data Dictionary 19
5.7 Collection and Parser Verification 19
6. STREAMING, CLASSIFICATION AND FAILURE HANDLING 19
6.1 Kafka-Based Messaging 19
6.2 Topic Design 19
6.3 Classification and Priority 19
6.4 Consumer Processing 20
6.5 Dead-Letter Queue Design 20
6.6 Error Handling and Traceability 20
6.7 Example Failure Observation 20
7. STORAGE, SEARCH AND SEMANTIC RETRIEVAL 20
7.1 Elasticsearch Document Indexing 20
7.2 Structured Search 20
7.3 Embedding Generation 21
7.4 Vector Storage 21
7.5 Retrieval Process 21
7.6 Structured and Semantic Search Compared 21
8. AI DIAGNOSIS AND RETRIEVAL-AUGMENTED GENERATION 21
8.1 Purpose of AI Diagnosis 21
8.2 RAG Pipeline Stages 21
8.3 Diagnosis Service 22
8.4 Diagnosis Content 22
8.5 AI Assistant 22
8.6 Human Review and Responsible Use 22
8.7 Separation of Diagnosis and Resolution 22
9. APPLICATION DESIGN AND DASHBOARD MODULES 22
9.1 Application Navigation 22

9.2 Overview Dashboard 22
9.3 Log Explorer 23
9.4 AI Log Assistant 23
9.5 AI Investigations 24
9.6 Incident Center 24
9.7 Dead Letter Queue 25
9.8 Dashboard Data Services 25
10. IMPLEMENTATION DETAILS 26
10.1 Project Structure 26
10.2 Runtime Configuration 26
10.3 Starting the Application 26
10.4 Service Availability Checks 26
10.5 Dashboard Loading Behaviour 26
11. TESTING AND VERIFICATION 27
11.1 Testing Strategy 27
11.2 Verification Cases 27
11.3 Integration Evidence 28
11.4 Test Limitations 28
12. RESULTS, DISCUSSION AND LIMITATIONS 28
12.1 Observed Dashboard Snapshot 28
12.2 Strengths of the Implementation 29
12.3 Current Limitations 29
12.4 Security Considerations 29
13. CONCLUSION AND FUTURE ENHANCEMENTS 29
13.1 Conclusion 29
13.2 Future Enhancements 30
REFERENCES 31
APPENDIX A — KAFKA TOPIC INVENTORY 32
APPENDIX B — PRIORITY INTERPRETATION 32
APPENDIX C — OPERATIONAL START-UP CHECKLIST 32
APPENDIX D — EVIDENCE AND REPRODUCIBILITY NOTE 32

LIST OF TABLES

Table	Description
2.1	Risk assessment
3.1	Functional requirements
3.2	Non-functional requirements
4.1	Major system components
5.1	Supported input formats
5.2	Canonical log fields
6.1	Severity-to-priority mapping
6.2	Kafka topics and purpose
10.1	Implementation modules
11.1	Verification and test cases
12.1	Observed dashboard snapshot

LIST OF FIGURES

Figure	Description	Page
4.1	Overall system architecture	14
4.2	Context-level data flow diagram	14
4.3	Level-1 data flow diagram	15
4.4	Canonical log record structure	15
4.5	Processing workflow	16
4.6	Use-case model	16
4.7	Logical data model	17
9.1	Overview dashboard	23
9.2	Log Explorer	23
9.3	AI Log Assistant	24
9.4	AI Investigations	24
9.5	Incident Center	25
9.6	Dead Letter Queue	25

1. INTRODUCTION

1.1 Background

Logs are chronological records of events produced by applications, operating systems, network devices and supporting services. They describe activities such as service startup, user requests, database operations, warnings, exceptions and infrastructure failures. When analysed effectively, logs help engineers understand system behaviour, identify abnormal patterns and reconstruct the sequence of events preceding an outage.

In a distributed application, each service may produce records in a different format and at a different rate. A single operational incident can generate messages across an API gateway, application service, message broker and database. An engineer may therefore need to search several files or tools, interpret inconsistent field names and manually compare timestamps before identifying a likely cause. This process becomes increasingly difficult as the number of services and the volume of events grow.

The project addresses this operational problem by creating a common processing path. It accepts several log formats, converts records into a canonical structure, applies validation and masking, streams events through Kafka, stores processed data in Elasticsearch and supports both conventional search and semantic investigation.

1.2 Problem Statement

The central problem is the absence of a unified and evidence-oriented workflow for handling heterogeneous application logs. Differences in syntax and field structure make it difficult to query records consistently. Invalid events can interrupt processing if they are not isolated. Sensitive values may appear in raw messages or structured fields. Keyword search alone may fail to find historical records that describe the same issue using different wording. Finally, the interpretation of large log collections requires time and operational expertise.

The proposed framework combines deterministic processing controls with search and AI-assisted diagnosis. It does not attempt to replace an engineer or an incident-management process; instead, it provides a consolidated view of log evidence and possible explanations.

1.3 Project Objectives

- Develop a collector that discovers log files and records collection progress through checkpoints.
- Parse supported input formats and transform valid events into a consistent canonical record.
- Validate required fields and isolate invalid records without silently treating them as successfully processed.
- Mask configured sensitive information before records are published for downstream use.
- Use Kafka topics to decouple collection, processing, indexing and AI diagnosis stages.
- Classify log events and assign an operational priority according to implemented rules.
- Index processed logs in Elasticsearch and provide structured filtering and search.
- Generate semantic embeddings and retrieve similar historical records from an Elasticsearch vector index.
- Use retrieved evidence as context for AI-generated diagnostic assistance.
- Provide a Streamlit interface for overview, search, investigations, incidents and DLQ inspection.

1.4 Scope

The scope includes file-based log collection, parsing and normalisation, validation, masking, Kafka-based streaming, rule-based classification, dead-letter handling, Elasticsearch indexing, semantic retrieval, RAG-based diagnosis and a multi-page dashboard. The implemented dashboard is read-only with respect to incident resolution and DLQ remediation. It displays diagnosis and failure evidence but does not acknowledge, resolve, delete, replay or retry these records.

1.5 Intended Users

The primary users are software developers, system administrators, site reliability personnel, support engineers and students studying distributed data-processing systems. The dashboard is intended to reduce the effort required to inspect records and understand possible relationships between current failures and historical events.

1.6 Organisation of the Report

The report begins with project planning and requirements, then explains the architecture and processing stages. Subsequent chapters describe storage, semantic retrieval, AI diagnosis and the dashboard. The final chapters document verification, observed behaviour, limitations and possible extensions.

2. PROJECT PLANNING AND FEASIBILITY

2.1 Development Approach

The system was developed incrementally. Core processing capabilities were implemented and tested before the dashboard was assembled around the available services. This approach allowed individual stages—collection, parsing, Kafka publishing, processing, indexing and retrieval—to be verified independently. The user interface was then connected to these services through dedicated data-access modules.

The project uses a modular architecture rather than a single monolithic script. Each major stage has a defined responsibility and communicates through files, Kafka topics or Elasticsearch indexes. This separation makes it easier to troubleshoot a failing stage without changing unrelated components.

2.2 Technical Feasibility

The project is technically feasible using widely available Python libraries and locally hosted infrastructure. Kafka and Elasticsearch are run in Docker containers, while the application and processing scripts execute in a Python virtual environment. Sentence Transformers provides local embedding generation. The language-model diagnosis stage uses a Groq client configured through environment variables.

2.3 Operational Feasibility

The Streamlit dashboard presents the main operational functions through a browser-based interface. Users can inspect summary metrics, filter log records, ask questions about failures, review generated investigations and inspect dead-letter messages. The application does require the underlying Kafka and Elasticsearch services to be available, and AI diagnosis additionally depends on valid model-provider configuration and network access.

2.4 Economic Feasibility

The implementation relies primarily on open-source software and locally run containers. This reduces licensing costs for the core processing, streaming, search and dashboard layers. The external language-model service may be subject to provider usage limits or pricing changes; therefore, the project should not be described as having zero operating cost in every deployment scenario.

2.5 Schedule and Deliverables

The work was organised around the following deliverables: (i) collection and checkpointing, (ii) parsing, validation and masking, (iii) Kafka publication and processing, (iv) Elasticsearch indexing and search, (v) embeddings and RAG diagnosis, and (vi) Streamlit dashboard integration and report preparation. The final implementation package contains source modules, configuration files, test scripts, sample log data and the dashboard pages.

2.6 Risk Assessment

Risk	Potential effect	Mitigation / control
Kafka unavailable	Events cannot move between pipeline stages.	Check broker health and topic metadata; expose connection errors.
Elasticsearch unavailable	Search, metrics and vector retrieval are unavailable.	Check service health and index configuration before dashboard use.
Malformed input	A record may not satisfy the canonical schema.	Validate fields and route failed events to a DLQ.
Sensitive values in logs	Confidential information may be exposed.	Apply masking to configured fields and text patterns before downstream processing.
LLM output is incorrect	An unsupported diagnosis may mislead an operator.	Display evidence and limitations; require human verification.
Provider or network failure	AI diagnosis may be delayed or unavailable.	Use bounded retry/failure handling and an AI diagnosis DLQ.

3. SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Purpose

This SRS defines the functional behaviour, external dependencies and quality requirements of the implemented log-processing framework. It provides a baseline for design, implementation and verification.

3.2 Product Perspective

The framework is a pipeline-oriented application consisting of a file collector, parser and normaliser, Kafka publisher, processing consumer, Elasticsearch indexer, semantic retrieval services, AI diagnosis consumer and Streamlit dashboard. It depends on Kafka for asynchronous messaging and Elasticsearch for document and vector storage.

3.3 Functional Requirements

ID	Requirement
FR-01	The system shall discover configured log files and read new content.
FR-02	The collector shall persist file identity and read-position checkpoints.
FR-03	The parser shall support JSON, XML, CSV, Syslog, CEF and LEEF input paths.
FR-04	The system shall create a canonical record with timestamp, level, service and message fields.
FR-05	The system shall validate required fields and report validation failures.
FR-06	The system shall mask configured sensitive keys and selected personal-data patterns.
FR-07	The system shall publish normalised events to Kafka.
FR-08	The processing service shall classify events and assign priority values.
FR-09	The system shall route failed processing events to a dead-letter topic.
FR-10	The indexer shall store processed records in Elasticsearch.
FR-11	The search layer shall support filtering by message, level, service and priority.
FR-12	The embedding layer shall generate vector representations for log content.
FR-13	The retrieval layer shall return semantically similar historical records.
FR-14	The AI layer shall produce a diagnosis using the current log and retrieved context.
FR-15	The dashboard shall expose overview, explorer, assistant, investigations, incidents and DLQ pages.

3.4 Non-functional Requirements

Quality attribute	Requirement
Performance	The pipeline should process records asynchronously and avoid requiring the dashboard to parse raw files on every page view.
Reliability	Checkpointing, Kafka delivery settings and DLQ routing should reduce data loss and isolate failures.
Maintainability	Processing responsibilities should remain separated into modules and services.

Scalability	Kafka partitions and consumer groups provide a basis for parallel processing as deployment capacity increases.
Security	Sensitive values should be masked; API credentials should be held in environment configuration rather than source code.
Usability	The dashboard should provide readable summaries, filters, tables and expandable record details.
Traceability	Kafka source coordinates and event metadata should be retained where available to support investigation.

3.5 External Interface Requirements

The system exposes a browser-based interface through Streamlit. Backend scripts communicate with Kafka using Python Kafka clients. Elasticsearch is accessed over its HTTP interface through the Elasticsearch Python client. AI diagnosis uses a Groq client, with provider credentials supplied through environment configuration. The local development environment uses Windows PowerShell and Docker Desktop.

3.6 Hardware and Software Environment

Component	Technology / configuration
Operating system	Windows development environment
Language	Python
Dashboard	Streamlit
Message broker	Apache Kafka 4.3.1 container
Search and vector store	Elasticsearch 8.17.4 container
Embedding model	Sentence Transformers: all-MiniLM-L6-v2 (384 dimensions)
Language model access	Groq client; credential loaded from environment configuration
Container runtime	Docker
Python environment	Project virtual environment (.venv)

3.7 Assumptions and Constraints

The collector assumes that source log files are accessible to the process and that the checkpoint directory is writable. Kafka and Elasticsearch must be running with compatible connection settings. The embedding model must be available to the Python environment. AI diagnosis depends on provider access and configured credentials. The current dashboard is designed for local development and is not, by itself, a hardened multi-tenant production deployment.

4. SYSTEM ANALYSIS AND DESIGN

4.1 Architectural Overview

The system follows a staged, event-driven processing architecture. Application log files are monitored by the Log Collector, which reads new complete records and passes them to the Log Parser. A supporting Checkpoint Manager maintains file identity and byte-position information so that collection can resume after a restart and react to file replacement or truncation. The Checkpoint Manager supports the collector; it is not a separate processing stage between collection and parsing. The parser masks configured sensitive values, maps supported input into a canonical record and validates the result. Valid records are published to the normalized-logs Kafka topic. A processing consumer assigns classification and priority and publishes the processed event. The Elasticsearch indexer stores processed events for structured search. In parallel, eligible ERROR and CRITICAL events are consumed by the AI diagnosis service, which uses semantic retrieval and the RAG pipeline before publishing diagnosis events for dashboard consumption.

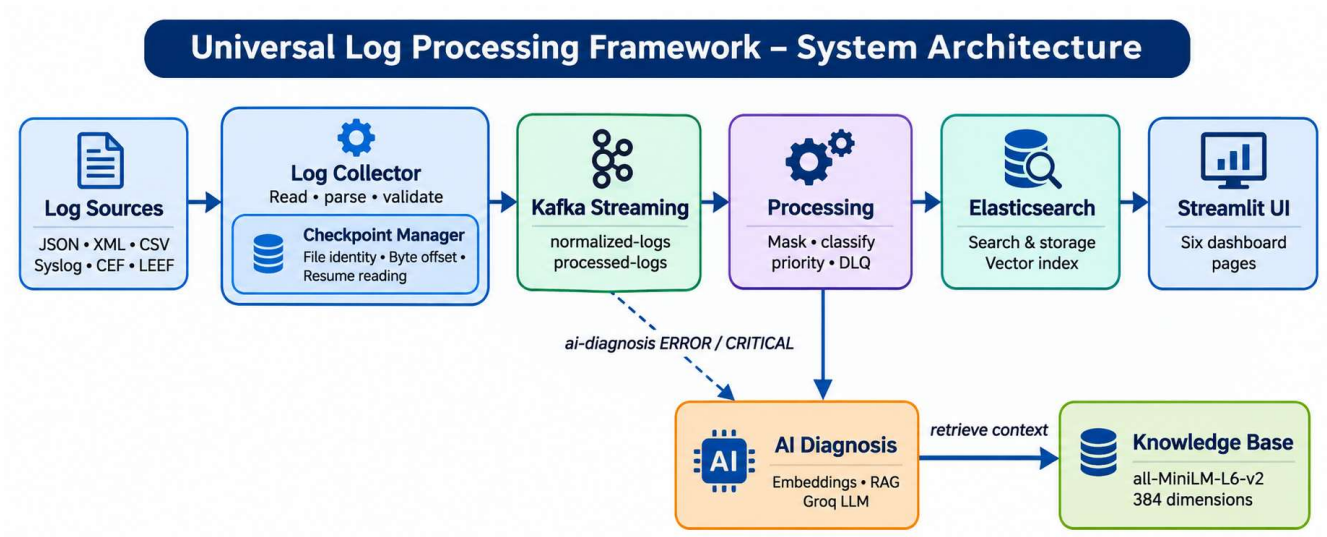


Figure 4.1. Overall system architecture of the implemented framework.

4.2 Context-Level Data Flow

At the context level, the framework receives log events from application log files and exposes processed information to an operator through the dashboard. Kafka provides the internal event transport. Elasticsearch stores searchable documents and semantic vectors. The AI provider is called by the diagnosis component when a diagnosis request is processed.

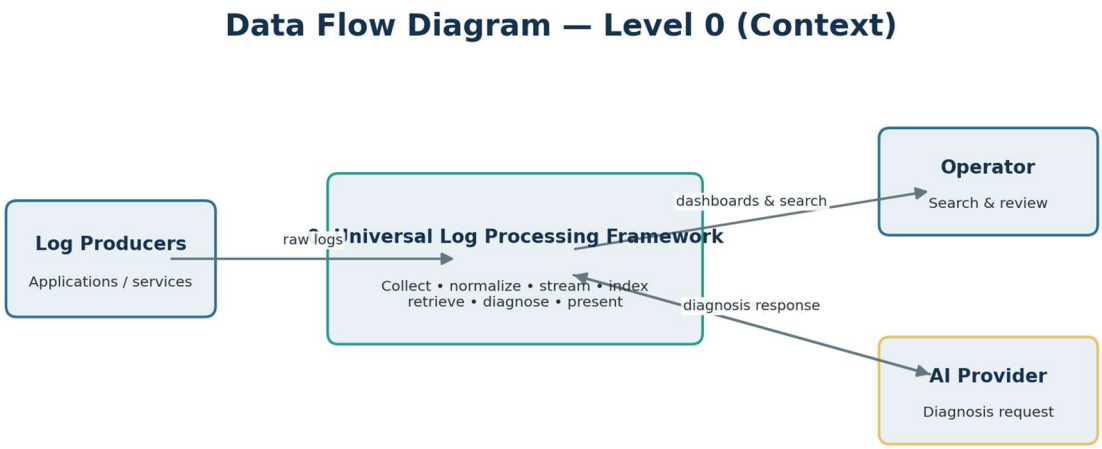


Figure 4.2. Context-level data flow diagram.

4.3 Level-1 Data Flow

The Level-1 view decomposes the system into collection, parsing and validation, streaming, processing, indexing, retrieval and presentation. Invalid events are separated from the normal path and written to a dead-letter stream. This separation is important because a malformed record should remain inspectable without being treated as a successfully processed log.

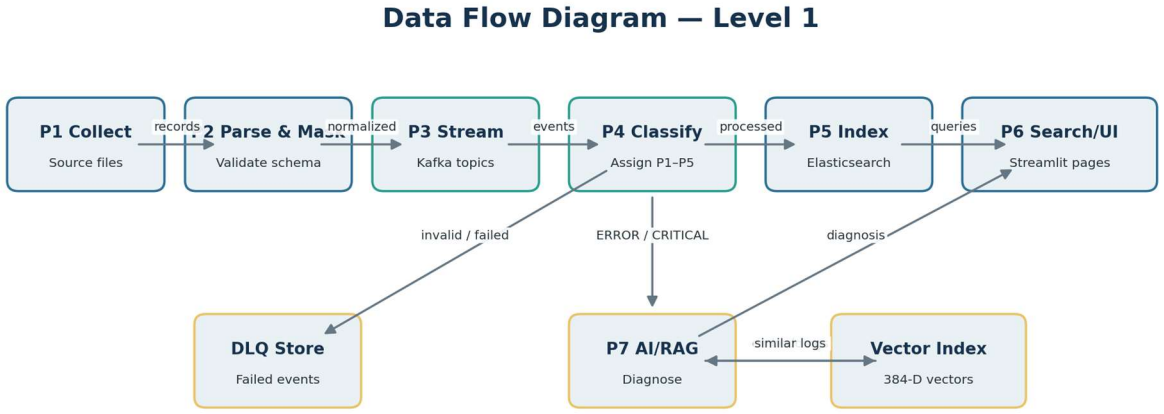


Figure 4.3. Level-1 data flow diagram.

4.4 Canonical Record Design

A canonical record provides a consistent structure regardless of the original log syntax. The core fields identified in the implementation include timestamp, level, service and message. The parser also supports metadata enrichment and preserves additional attributes where appropriate. The canonical representation allows downstream services to apply common classification, indexing and search logic.

Canonical Log Record — Illustrative Example

```
{
  "timestamp": "2026-11-08T08:30:00Z",
  "level": "ERROR",
  "service": "payment-service",
  "message": "Database connection failed",
  "attributes": {"request_id": "req-1042"},
  "metadata": {"source_format": "JSON"}
}
```

Figure 4.4. Canonical log record structure.

4.5 Processing Workflow

The workflow begins when the collector discovers a file and checks its checkpoint. New content is read, parsed and normalised. A valid event is published to Kafka; an invalid event is saved through the failure path. The processing service consumes normalised events, applies the classification rules and publishes processed records. The indexer stores the result in Elasticsearch. Eligible error records are also considered by the AI diagnosis service.

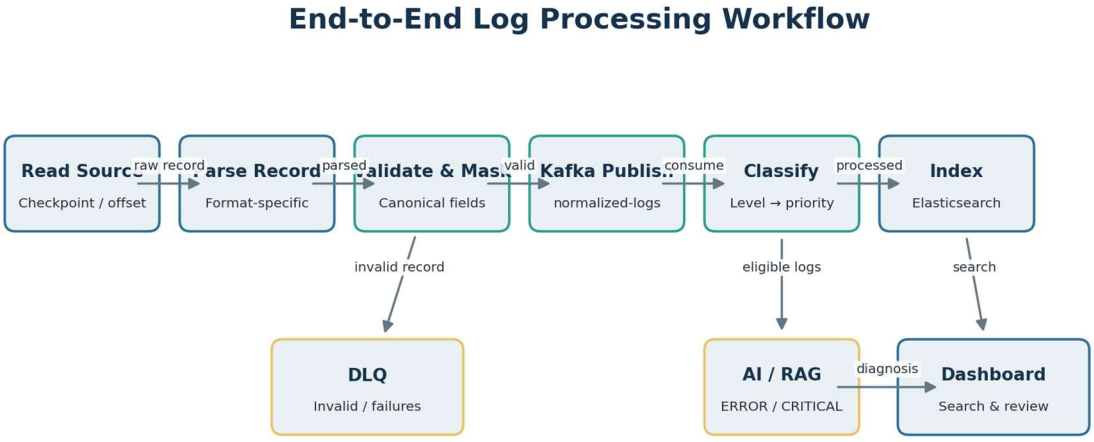


Figure 4.5. End-to-end processing workflow.

4.6 Use-Case Model

The principal actor is the operator or developer who uses the dashboard. The actor can inspect summary metrics, search and filter logs, ask the AI assistant a question, review diagnosis records, inspect incident candidates and examine dead-letter messages. Pipeline services perform collection, processing, indexing and diagnosis without requiring the operator to invoke each internal step manually.

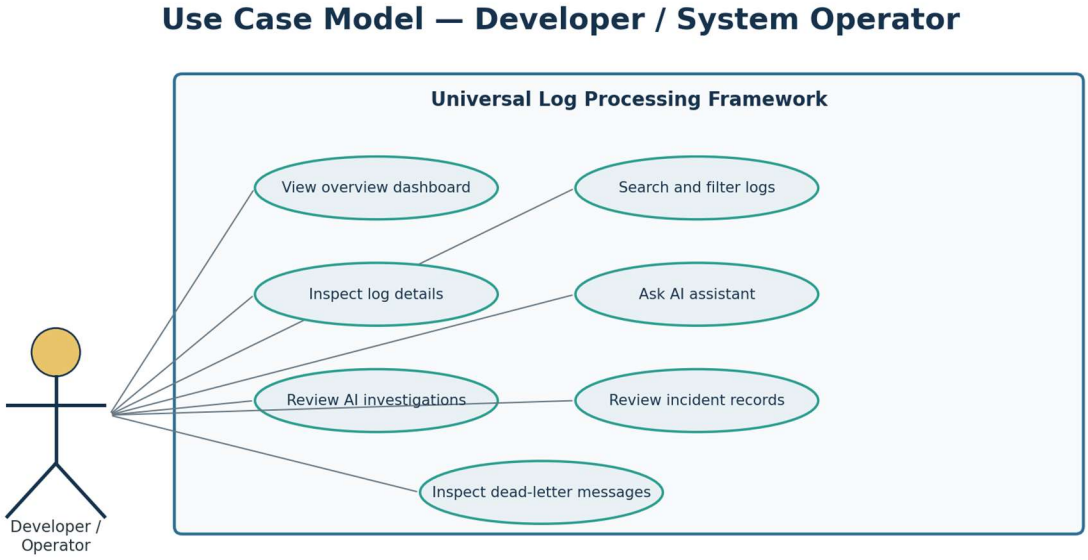


Figure 4.6. Use-case model for the log framework.

4.7 Logical Data Model

The logical data model describes the relationship between source log events, normalised records, processed records, diagnosis events and dead-letter records. Kafka topic coordinates provide provenance for streamed messages. Elasticsearch stores processed log documents and a separate vector knowledge base supports semantic retrieval.

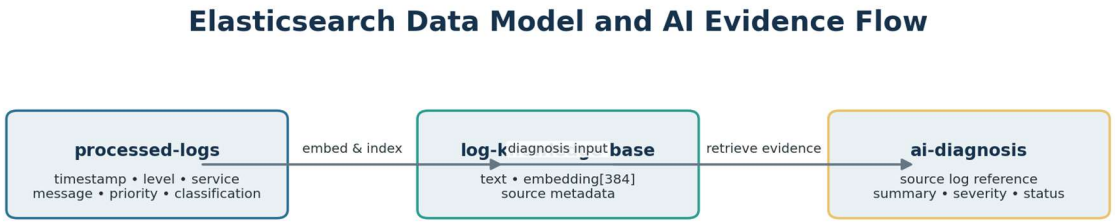


Figure 4.7. Logical data model.

4.8 Major Components

Component	Responsibility	Primary interface
Log collector	Discover files, track file identity and checkpoint offsets.	File system / checkpoint JSON
Parser and normaliser	Parse, mask, enrich, canonicalise and validate.	Python functions / JSON records
Kafka publisher	Serialise and publish events.	Kafka producer
Processing service	Classify and assign priority; route failures.	normalized-logs → processed-logs / DLQ
Elasticsearch indexer	Create/check index and index processed events.	processed-logs → Elasticsearch
Search service	Query indexed documents using structured filters.	Elasticsearch query API
Embedding and vector store	Generate embeddings and store/search vectors.	384-dimensional vectors
RAG and diagnosis	Retrieve context and request an AI diagnosis.	Retriever + Groq client
Streamlit dashboard	Present metrics, records and investigation views.	Browser UI

5. LOG COLLECTION AND NORMALISATION

5.1 File Discovery and Collection

The collector is implemented in log_collector.py. Its functions include file discovery, file identity generation, checkpoint lookup, checkpoint loading and saving, and collection of new records. The collector uses the stored identity and read position to determine which part of a file has already been processed. This supports incremental collection and helps prevent the same file content from being emitted repeatedly after a normal restart.

Checkpoint state is stored as JSON metadata under the project checkpoint directory. The collector also contains separate paths for saving failed and normalised records. File rotation is considered through file identity and discovery logic, allowing the collector to distinguish file instances rather than relying only on a filename.

The repository's parser implementation and focused tests cover JSON, XML, CSV, simple application text, Python-style application text, Syslog, Common Event Format (CEF) and Log Event Extended Format (LEEF). The security-event formats include Syslog RFC 5424 and RFC 3164, CEF records with epoch-millisecond timestamps, LEEF 1.0 tab-delimited extensions and LEEF 2.0 custom-delimiter extensions. XML parsing extracts values from element attributes and child elements. The tested CSV reader expects one headerless physical line with the columns timestamp, level, service and message; additional columns are retained as attributes. CSV headers and multiline quoted CSV records are outside this line-oriented collector's current supported input contract.

5.2 Supported Input Formats

Format	Typical representation	Parsing purpose
JSON	Structured key-value object or JSON record.	Read named fields and nested attributes.
XML	Element-based document.	Extract values from XML nodes and attributes.
CSV	Headerless, single-line record.	Map positional fields to canonical fields.
Syslog	Timestamp, host, process and message text.	Interpret conventional system event lines.
CEF	Common Event Format header and extension.	Extract event metadata and extension fields.
LEEF	Log Event Extended Format header and attributes.	Interpret vendor-style structured security events.

Format-specific parsing is routed through parse_by_format, while parse_by_source selects a parser using the configured source-to-format mapping. The parser normalizes supported timestamps, maps format-specific severity information into the framework's accepted levels, retains additional source fields in attributes and records source and format metadata. For Syslog, the PRI value is checked against the RFC range 0–191 before its severity component is mapped. RFC 3164 messages do not carry a year in the conventional timestamp; the current parser infers the year at processing time and records that inference in the event attributes. These format-specific behaviors are covered by the repository's focused parser tests.

5.3 Parsing and Canonicalisation

The parser is implemented in log_parser.py. The source exposes format-specific parsing functions as well as general dispatch functions such as parse_any_log, parse_by_format and parse_by_source. Parsed values are passed to canonical-record construction so that downstream components can work with a stable set of field names.

The canonical record requires timestamp, level, service and message. The accepted log-level vocabulary includes INFO, WARNING, ERROR, DEBUG and CRITICAL. Additional attributes can be retained as metadata, allowing source-specific context to remain available without changing the common processing contract.

The parser also supports format-specific extraction and validation for XML, CSV, Syslog, CEF and LEEF records. The XML parser extracts root attributes and child-element values. The CSV parser processes one physical line at a time, using timestamp, level, service and message as the primary fields, while retaining additional columns through generated field names such as field_5. The current CSV collector contract does not support header rows or multiline quoted fields. Syslog parsing supports RFC 5424 and RFC 3164 formats, with PRI validation and severity mapping. The CEF parser supports epoch-millisecond timestamps, while LEEF 1.0 supports tab-delimited extensions and LEEF 2.0 supports custom delimiters. These format-specific behaviours are covered by focused parser tests.

5.4 Validation

Validation is performed after the canonical record is built. The parser provides detailed and boolean validation functions. A record that lacks a required field is treated as invalid and is directed to the failure-handling path. For example, the captured DLQ sample contains the error “Missing required field: timestamp”; the original event included service and level information but did not contain a timestamp.

5.5 Sensitive-Data Masking

The parser includes text masking, raw-event masking and sensitive-data masking functions. The configured sensitive-key set includes password, passwd, secret, token, access_token, refresh_token, api_key, authorization, cookie, credit_card, card_number and ssn. The implementation also includes masking for email addresses and phone-like sequences in text. Masking is applied as part of the parsing and normalisation workflow so that downstream services do not need to repeatedly interpret the original raw structure.

5.6 Canonical Data Dictionary

Field	Meaning	Requirement / use
timestamp	Time associated with the log event.	Required; supports ordering and time-based analysis.
level	Severity level supplied or inferred from the source.	Required; used for classification and filtering.
service	Application or component that generated the event.	Required; used for service-level analysis.
message	Human-readable event description.	Required; used for search and semantic text.
metadata / additional attributes	Source-specific or enriched information.	Optional; preserves useful context.

5.7 Collection and Parser Verification

Earlier development checks confirmed that checkpoint state could be saved and loaded, a repeated collection call did not re-emit previously consumed content, and a simulated restart using the stored checkpoint did not create a duplicate output record. The latest repository README records an additional parser verification run on 10 November 2026: test_parser_errors.py reported 7/7 checks passed, and test_parser_formats.py reported 18/18 checks passed, for a combined 25/25 parser checks. The format suite checks successful XML, CSV, Syslog RFC 5424/RFC 3164, CEF, LEEF 1.0 and LEEF 2.0 parsing; invalid-input rejection; and source-based routing for XML, CSV, Syslog, CEF and LEEF. The error suite checks unknown sources, malformed JSON, missing required fields, invalid severity, valid simple-text records and valid JSON. These results are repository-recorded local test results, not a claim of exhaustive production testing.

6. STREAMING, CLASSIFICATION AND FAILURE HANDLING

6.1 Kafka-Based Messaging

Apache Kafka is used as the event transport layer between independent pipeline stages. Producers publish records to topics, while consumer services subscribe to the topics they are responsible for processing. This design separates collection from downstream processing and permits consumers to operate independently.

The local development environment uses an Apache Kafka 4.3.1 Docker container exposed on port 9092. The project source uses kafka-python and confluent-kafka clients in different modules. The publisher module defines a LogKafkaPublisher abstraction and serialises Python values as JSON before publishing.

6.2 Topic Design

Topic	Role in the pipeline
normalized-logs	Receives canonical records from the collection and parsing stage.
processed-logs	Receives records after processing and classification.
dead-letter-logs	Stores events rejected by the log-processing path.
ai-diagnosis	Carries completed AI diagnosis events for investigation and incident views.
ai-diagnosis-dlq	Stores failures from the AI diagnosis processing path.
__consumer_offsets	Kafka-managed internal topic used to store consumer offset information.

6.3 Classification and Priority

The log-processing service consumes normalised events and applies deterministic rules to map the event level to a classification and priority. The implemented mapping is summarised below. Priority labels are operational categories used by the application; they should not be interpreted as a universally standard severity model.

Input level	Classification	Priority
CRITICAL	SYSTEM_FAILURE	P1
ERROR	APPLICATION_ERROR	P2
WARNING	POTENTIAL_ISSUE	P3
INFO	NORMAL_OPERATION	P4
DEBUG	DIAGNOSTIC_INFORMATION	P5

6.4 Consumer Processing

The processing service reads from normalized-logs and publishes accepted output to processed-logs. Its configuration uses a named consumer group and manual offset commits. Manual commit control allows the service to decide when a message is considered processed, rather than committing progress automatically before application-level handling has completed.

6.5 Dead-Letter Queue Design

A dead-letter queue (DLQ) is a separate Kafka topic used to preserve messages that could not be processed successfully. The framework has two distinct failure destinations: dead-letter-logs for the log-processing stage and ai-diagnosis-dlq for the AI diagnosis stage. The dashboard reads both topics and presents failure metadata, including the failure reason and source topic coordinates when those values are available.

The DLQ dashboard is deliberately read-only. It does not delete, replay, retry or modify Kafka messages. This boundary is important because retrying or replaying a failed message requires an explicit operational policy, including validation of the original cause and protection against duplicate side effects.

6.6 Error Handling and Traceability

The DLQ reader normalises different payload shapes and extracts fields such as failure time, processor, failure reason, source topic, source partition and source offset. It also decodes the original message when it is stored as JSON text. Kafka topic, partition and offset values are retained for the DLQ record itself. These details help an operator locate the event in the original stream and understand which stage rejected it.

6.7 Example Failure Observation

A development command executed against the DLQ service retrieved three retained records in approximately 0.65 seconds. The returned sample showed a log-processing failure caused by a missing timestamp in an event on normalized-logs. This is a single observed local run and should not be treated as a throughput benchmark or a guarantee of response time under load.

7. STORAGE, SEARCH AND SEMANTIC RETRIEVAL

7.1 Elasticsearch Document Indexing

Elasticsearch is used as the document store and search engine for processed log records. The indexer consumes processed-logs, checks or creates the target index and writes the records as searchable documents. Stable document identifiers are derived from Kafka source coordinates in the implementation, which supports traceability and reduces the risk of duplicate indexing when the same source event is encountered again.

The local environment uses Elasticsearch 8.17.4 in Docker, exposed on port 9200. The dashboard data service queries the processed-logs index for counts, summary metrics, recent records and daily activity.

7.2 Structured Search

The ElasticsearchLogSearch class provides methods for searching all records and filtering by level, service, priority and message. The Log Explorer page exposes these dimensions through sidebar controls. It also supports pagination and displays the matching records in a table, with an inspection area for record details.

7.3 Embedding Generation

Keyword search compares the terms supplied by a user with indexed text. Semantic retrieval addresses a different problem: two records may describe a similar failure even when their wording is not identical. The LogEmbeddingGenerator class builds a text representation from relevant log attributes and generates a dense vector using the all-MiniLM-L6-v2 Sentence Transformer model.

The configured embedding dimension is 384. The embedding is a numerical representation of the text that can be compared with other vectors using a similarity measure. It is not a diagnosis by itself; it is used to retrieve records that may provide relevant historical context.

7.4 Vector Storage

The ElasticsearchVectorStore class manages the vector index and provides operations to create the index, index a log, search for similar records, delete a record, count records and close the client. The vector index is named log-knowledge-base and uses a dense-vector field configured for cosine similarity.

7.5 Retrieval Process

The LogRetriever class converts a search query into an embedding and requests similar records from the vector store. The RAG pipeline constructs a query from the current event, including service, level, classification and message information, then retrieves a limited set of historical records. The default retrieval depth documented in the project context is three records.

7.6 Structured and Semantic Search Compared

Aspect	Structured search	Semantic retrieval
Matching basis	Field values, filters and text terms.	Similarity between embedding vectors.
Best use	Find a known service, level, priority or phrase.	Find historical records describing a related issue.
Output	Records satisfying query conditions.	Nearest records according to vector similarity.
Limitation	May miss wording variations.	May return related but not causally identical records.

8. AI DIAGNOSIS AND RETRIEVAL-AUGMENTED GENERATION

8.1 Purpose of AI Diagnosis

The AI diagnosis layer is intended to assist an operator in interpreting error and critical events. Rather than sending an isolated message to a language model without context, the application retrieves similar historical log records and supplies them as supporting evidence. This approach is known as retrieval-augmented generation (RAG).

8.2 RAG Pipeline Stages

1. Receive an eligible current log event from the processed-log stream.
2. Construct a retrieval query from the event fields, including service, level, classification and message.
3. Generate a query embedding using the configured Sentence Transformer model.
4. Search the Elasticsearch vector index for semantically similar historical records.
5. Build a bounded context containing the current event and retrieved records.
6. Submit the context and diagnostic instructions to the Groq language-model client.
7. Parse the model response into the diagnosis structure expected by downstream services.
8. Publish the diagnosis event to ai-diagnosis; route exhausted failures to ai-diagnosis-dlq.

8.3 Diagnosis Service

The ai_diagnosis_service.py module consumes processed logs and is configured to analyse ERROR and CRITICAL events. The service uses a bounded retry policy with a maximum of three attempts in the documented implementation. It creates a source identifier from Kafka topic, partition and offset, allowing the diagnosis output to retain a reference to the originating processed event.

8.4 Diagnosis Content

The diagnosis structure can include a problem summary, possible causes, supporting evidence, verification steps, recommended actions, confidence information and limitations. The exact fields available depend on the model response and response parser. The dashboard displays the returned information without converting the model output into a confirmed operational fact.

8.5 AI Assistant

The separate AI Assistant service distinguishes general conceptual questions from investigation-related questions. For investigation questions, it can retrieve historical logs and provide the evidence alongside the answer. The Streamlit page maintains conversation messages in session state and allows the user to inspect retrieved historical records, including service, severity, classification, message and similarity score when present.

8.6 Human Review and Responsible Use

A language model can produce plausible but incorrect explanations. Retrieved historical records can also be similar in wording without sharing the same root cause. For these reasons, the AI diagnosis should be treated as decision support. Operators should verify the proposed cause against current metrics, traces, configuration, deployment history and direct service checks before taking action.

8.7 Separation of Diagnosis and Resolution

The Incident Center is derived from AI diagnosis events. A diagnosis status of COMPLETED means the diagnosis-processing task completed; it does not mean that the underlying service incident has been fixed. The dashboard explicitly communicates this distinction and remains read-only with respect to incident acknowledgement and resolution.

9. APPLICATION DESIGN AND DASHBOARD MODULES

9.1 Application Navigation

The Streamlit application is launched from the project root with the command `streamlit run frontend/app.py`. The application uses a wide layout and provides six main dashboard areas. The screenshots in this chapter are captured from the running application on 8 November 2026 and represent a development snapshot, not a permanent system baseline.

9.2 Overview Dashboard

The Overview Dashboard presents high-level visibility into processed logs and system activity. It displays total processed logs, error logs, warning logs and critical/P1 logs. It also visualises the distribution of log severity and priority and includes recent activity information. In the captured snapshot, the dashboard displayed 430 processed logs, 126 error logs, 127 warning logs and zero critical/P1 logs. These values describe the retained data visible at capture time.

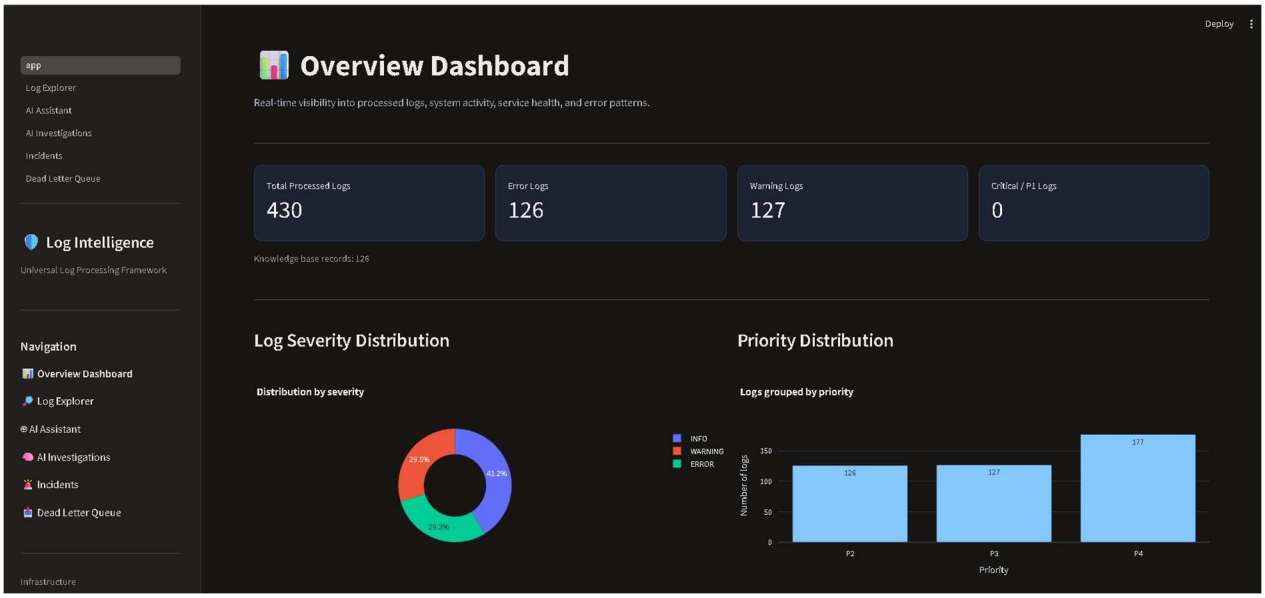


Figure 9.1. Overview Dashboard with summary metrics and distribution charts.

9.3 Log Explorer

The Log Explorer provides a searchable view of processed records. Its filters include message keyword, log level, priority, service and classification. The page displays matching-log counts, the number of errors on the current page, affected services and pagination information. The record table includes timestamp, service, level, priority, classification and message, with a separate area for inspecting selected record details.

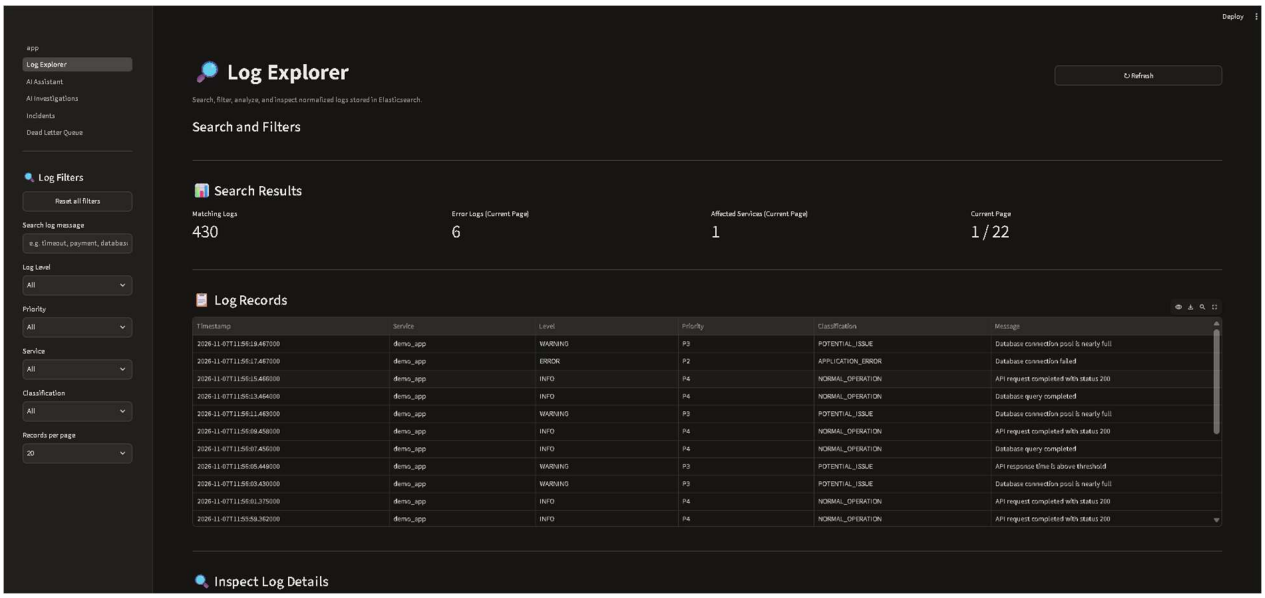


Figure 9.2. Log Explorer with sidebar filters and searchable record table.

9.4 AI Log Assistant

The AI Log Assistant provides a conversational interface for asking questions about application failures, services and log patterns. The page retains the conversation in session state and displays retrieved historical log evidence with the response. A clear-conversation control resets the visible chat history and associated evidence.

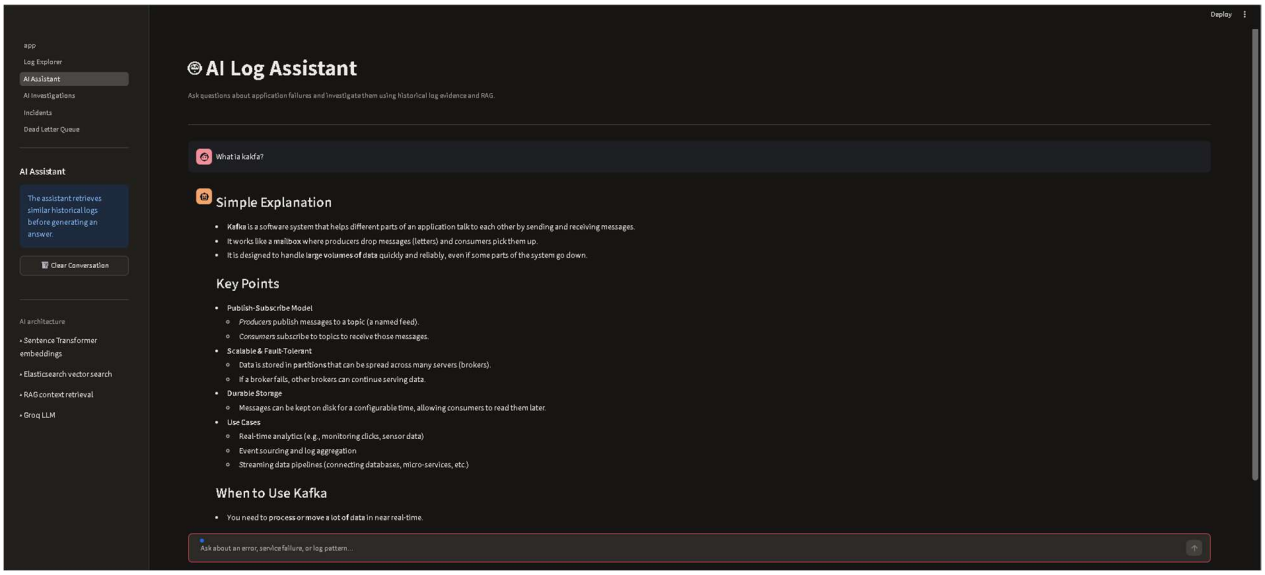


Figure 9.3. AI Log Assistant showing a conceptual answer and chat interface.

9.5 AI Investigations

The AI Investigations page reads diagnosis events from the ai-diagnosis Kafka topic. It provides service, severity and text filters; summary metrics; an investigation table; and expandable diagnosis details. The captured screenshot shows 129 retained investigations, zero critical investigations, 129 high-severity investigations and four affected services. These counts are specific to the retained topic contents at the time of capture.

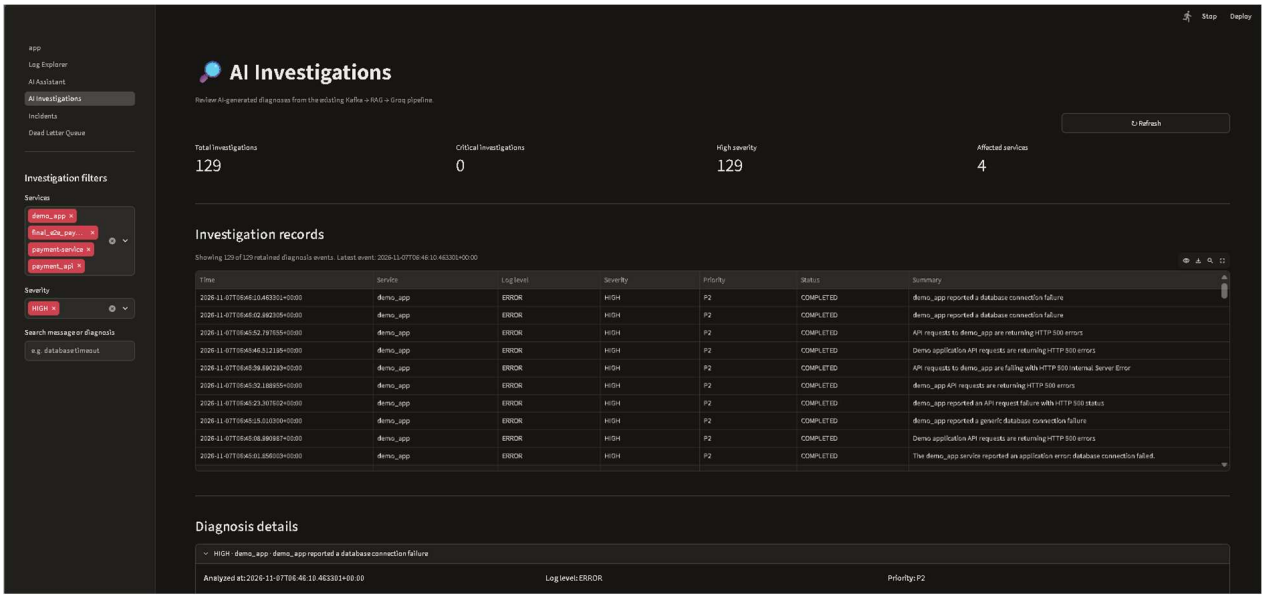


Figure 9.4. AI Investigations page showing diagnosis records and filters.

9.6 Incident Center

The Incident Center derives incident candidates from AI diagnosis events. It displays total records, critical and high severity counts, affected services and a table of incident summaries. Expanded details include the incident reference, original message, AI problem summary, possible causes, supporting evidence, verification steps, recommended actions and limitations when returned by the diagnosis data. The page is read-only and does not mark incidents as resolved.

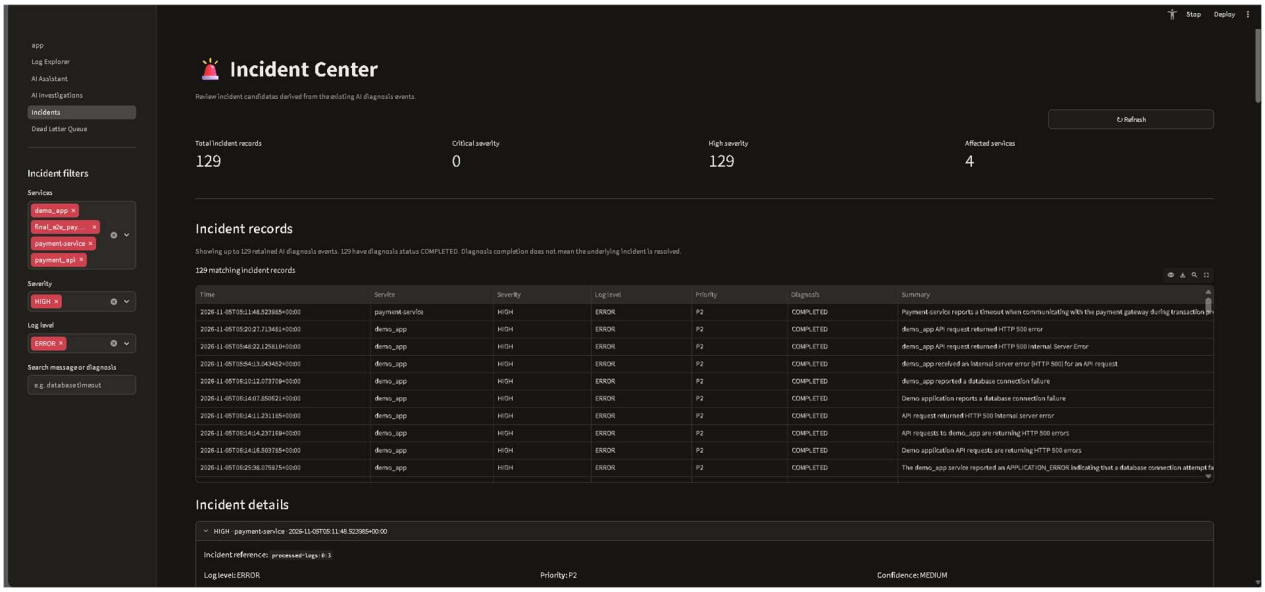


Figure 9.5. Incident Center with incident records and diagnosis details.

9.7 Dead Letter Queue

The Dead Letter Queue page reads retained messages from dead-letter-logs and ai-diagnosis-dlq. It presents total failed messages, counts by DLQ type and affected processors. Filters allow the operator to select topics and processors and search failure reasons or payload content. The record table includes failure time, DLQ topic, processor, failure reason and source coordinates. Expandable details expose the original record and complete DLQ payload.

In the captured snapshot, the page displayed three retained failures: three from the log-processing DLQ and none from the AI-diagnosis DLQ. The failure reason shown in the sample was a missing required timestamp. The page states that it is read-only and that displayed messages depend on Kafka topic retention.

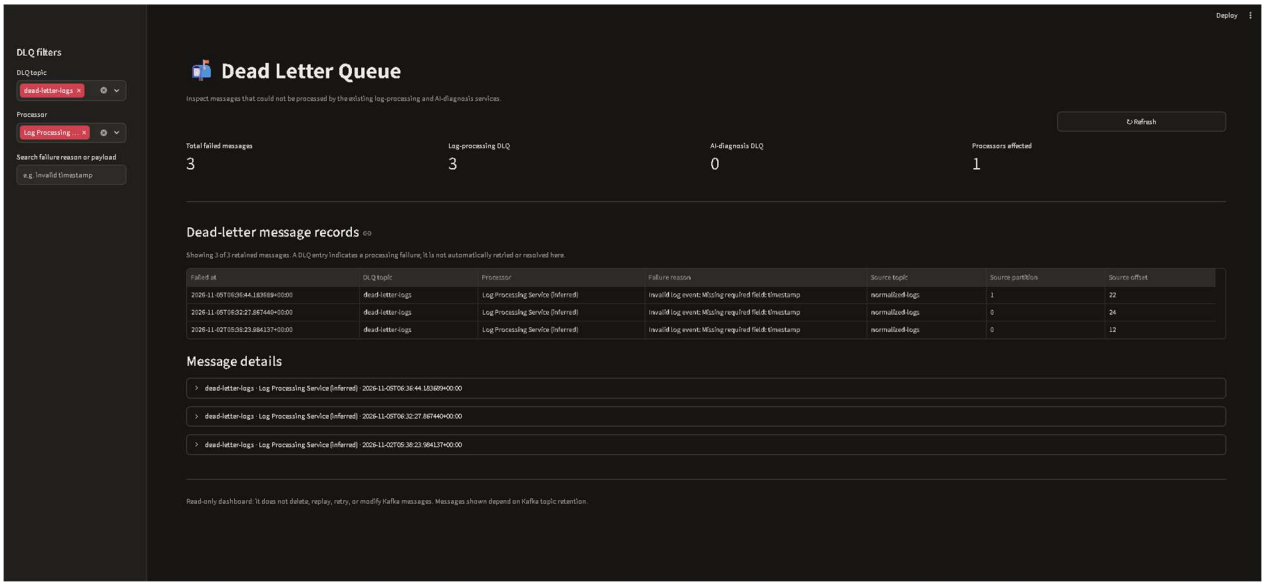


Figure 9.6. Dead Letter Queue showing retained failures and message metadata.

9.8 Dashboard Data Services

The dashboard separates presentation code from data-access logic. The overview uses DashboardDataService; AI chat uses AIAssistantService; investigations use InvestigationsService; incident records use

IncidentsService; and DLQ records are obtained through the dedicated DLQ service. This separation improves maintainability and allows the UI pages to focus on rendering and interaction.

10. IMPLEMENTATION DETAILS

10.1 Project Structure

File / directory	Implementation responsibility
log_collector.py	File discovery, checkpointing and incremental collection.
log_parser.py	Parsing, canonical record construction, masking and validation.
kafka_publisher.py	JSON serialisation and Kafka publication abstraction.
log_processing_service.py	Kafka consumer, classification, priority and processing failure handling.
elasticsearch_indexer.py	Create/check processed-log index and index Kafka records.
elasticsearch_search.py	Structured Elasticsearch search functions.
log_embedding_generator.py	Text preparation and Sentence Transformer embeddings.
elasticsearch_vector_store.py	Vector index creation, indexing and similarity search.
log_retriever.py	Retrieval of similar historical log records.
rag_context_builder.py	Construction of bounded context for AI requests.
rag_pipeline.py	Coordinates retrieval, context building and diagnosis.
llm_diagnosis.py	Prompt construction, model request and response parsing.
ai_diagnosis_service.py	Kafka consumer for AI diagnosis and AI DLQ handling.
frontend/app.py	Overview dashboard entry point.
frontend/pages/*.py	Five additional Streamlit pages.
frontend/*_service.py	Data access and application services used by dashboard pages.

10.2 Runtime Configuration

The project uses environment configuration for service endpoints and secret credentials. The .env file is excluded from version control by the project .gitignore. The report intentionally does not reproduce API keys, tokens, passwords or other secret values. Local Docker services are accessed through localhost endpoints from the host Python environment.

10.3 Starting the Application

The Streamlit application is started from the project root after activating the project virtual environment. Kafka and Elasticsearch containers must be running before opening pages that depend on those services.

```
streamlit run frontend/app.py
```

The application is normally available at http://localhost:8501. If the import system cannot resolve the frontend package, launch the command from the project root directory so that the project package is on the Python import path.

10.4 Service Availability Checks

Elasticsearch availability can be checked by requesting its root endpoint at http://localhost:9200 and its cluster health endpoint. A one-node development cluster may report yellow health because replica shards cannot be allocated to another node; this is different from a failed primary shard. Kafka availability can be checked by listing the configured topics through the Kafka command-line utility inside the container.

10.5 Dashboard Loading Behaviour

The investigations and incident pages read retained diagnosis events from Kafka, while the DLQ page reads retained messages from two DLQ topics. The DLQ service uses a read-only consumer configuration, inspects topic metadata and watermarks, and reads a bounded recent window from each available partition. The dashboard applies caching or session-state retention to reduce unnecessary repeated reads. Long loading can

still occur when a broker is unavailable, topic metadata is slow, partitions contain large retained windows, or a page performs a fresh read after a refresh action. The captured local verification for fetch_dead_letter_records returned three records in 0.65 seconds, but this observation is not a universal performance guarantee.

11. TESTING AND VERIFICATION

This chapter records the checks used to verify the main processing stages and dashboard functions. It also distinguishes results observed during local development from checks supported only by configuration or screenshot evidence.

11.1 Testing Strategy

Testing was performed at multiple levels during development. Unit-style checks were used for parsing, validation, masking, checkpoint operations and individual service functions. Integration checks verified communication with Kafka and Elasticsearch. End-to-end checks followed records through collection, processing, indexing and AI diagnosis. Dashboard verification was performed by opening each Streamlit page and inspecting its rendered data.

The repository also contains a GitHub Actions workflow at .github/workflows/parser-tests.yml. It is configured to run the two parser test scripts on pushes and pull requests to the main branch, using Python 3.11 on Ubuntu. The workflow file was present when this report was prepared; a successful hosted run was not verified.

11.2 Verification Cases

ID	Test	Procedure	Expected result	Evidence / status
TC-01	Collector checkpoint save/load	Save checkpoint state and reload it.	Checkpoint metadata is available after reload.	Previously reported as passed during development.
TC-02	Duplicate collection prevention	Run collection twice against unchanged input.	Second run does not re-emit already collected content.	Previously reported as passed during development.
TC-03	Restart continuity	Reload checkpoint after simulated restart.	No duplicate output is created for already processed content.	Previously reported as passed during development.
TC-04	Required-field validation	Submit an event without timestamp.	Validation fails with a missing timestamp reason.	Observed in retained DLQ record.
TC-05	Sensitive-key masking	Submit password and nested token fields.	Configured sensitive values are masked.	Previously reported as passed during development.
TC-06	Kafka topic availability	List configured Kafka topics.	Expected application topics are present.	Topic list supplied in project session.
TC-07	Elasticsearch connectivity	Request Elasticsearch root endpoint.	Cluster returns version and node information.	Observed in project session.
TC-08	DLQ reader integration	Call fetch_dead_letter_records().	Return retained records with failure and source metadata.	Three records returned in 0.65 seconds in one local run.
TC-09	Dashboard overview	Open the overview page.	Summary metrics and charts render.	Screenshot supplied.
TC-10	Log Explorer	Open page and inspect filters/table.	Records and filter controls render.	Screenshot supplied.
TC-11	AI Assistant	Ask a conceptual question.	Assistant response and chat interface render.	Screenshot supplied.

TC-12	AI Investigations	Open investigation page.	Retained diagnosis records and details render.	Screenshot supplied.
TC-13	Incident Center	Open incident page.	Incident candidates and diagnosis details render.	Screenshot supplied.
TC-14	Dead Letter Queue	Open DLQ page.	Retained failure records and payload details render.	Screenshot supplied.
TC-15	Parser error and source validation suite	Run python test_parser_errors.py.	Seven checks pass, covering unknown source, malformed JSON, missing fields, invalid severity and valid JSON/simple-text records.	Repository README records 7/7 passed on 10 November 2026.
TC-16	Multi-format parser suite	Run python test_parser_formats.py.	Eighteen checks pass across XML, CSV, Syslog RFC 5424/RFC 3164, CEF, LEEF 1.0/2.0, invalid-input rejection and source routing.	Repository README records 18/18 passed on 10 November 2026.
TC-17	Parser CI workflow configuration	Inspect .github/workflows/parser-tests.yml.	Workflow is configured for pushes and pull requests to main, using Python 3.11 and running both parser scripts.	Workflow configuration is present in GitHub; successful hosted-run status not verified in this report.

Across the two focused parser suites, 25 checks passed: 7 covered error handling and 18 covered supported formats and source routing. These were local results recorded on 10 November 2026; they are not a production-scale performance assessment.

11.3 Integration Evidence

The project session includes evidence that the Kafka topics were listed, Elasticsearch responded with version 8.17.4 and cluster health information, and the DLQ reader returned three records with their failure and source metadata. The dashboard screenshots provide visual evidence that all six pages rendered with retained data. These observations are reported as captured development evidence; they do not constitute a formal load, stress, penetration or disaster-recovery test.

11.4 Test Limitations

The available evidence does not establish throughput under sustained high load, maximum supported log size, multi-node failover behaviour, security penetration resistance or the statistical accuracy of AI diagnoses. Those areas require controlled test datasets, defined acceptance thresholds and repeatable test execution. Where a test was not supported by captured evidence, this report does not label it as passed.

12. RESULTS, DISCUSSION AND LIMITATIONS

12.1 Observed Dashboard Snapshot

Dashboard area	Observed values in supplied screenshot	Interpretation
Overview	430 processed logs; 126 errors; 127 warnings; 0 critical/P1.	A retained-data snapshot of the indexed processed-log collection.

Log Explorer	430 matching logs; 22 pages at 20 records per page.	Search and pagination controls were visible.
AI Investigations	129 records; 0 critical; 129 high; 4 affected services.	Retained diagnosis events were available for review.
Incident Center	129 incident records; 0 critical; 129 high; 4 affected services.	Incident candidates were derived from diagnosis records.
Dead Letter Queue	3 failures; 3 log-processing DLQ; 0 AI diagnosis DLQ; 1 processor.	Three retained validation failures were visible.

The values above are not intended to be reconciled as a single simultaneous system total. Each dashboard page reads a different data source or retained event collection, and screenshots may have been captured at different times. Counts can change as new logs arrive and as Kafka retention or Elasticsearch indexing changes.

12.2 Strengths of the Implementation

- The processing stages are separated into modules and Kafka topics.
- Multiple common log formats are handled through a shared canonical record.
- Checkpointing supports incremental collection and restart continuity.
- Validation failures are retained in dedicated dead-letter topics.
- Structured search and semantic retrieval are both available.
- AI-generated diagnoses can be inspected together with retrieved historical evidence.
- The dashboard provides separate operational views for logs, investigations, incidents and DLQ records.

12.3 Current Limitations

- The project is demonstrated in a local development environment; production deployment hardening is outside the current scope.
- The dashboard is read-only and does not provide incident acknowledgement, resolution, DLQ replay or retry workflows.
- The diagnosis quality depends on the retrieved historical records, prompt design and language-model response.
- The AI diagnosis service is dependent on external model-provider availability and credentials.
- The current evidence does not include a formal performance benchmark or a large-scale stress test.
- Kafka and Elasticsearch availability are prerequisites for the corresponding dashboard functions.

12.4 Security Considerations

The parser masks a defined set of sensitive keys and selected text patterns. The project excludes .env from version control, which is an important credential-management practice. A production deployment would additionally require transport encryption, authenticated Kafka and Elasticsearch access, least-privilege service accounts, secret rotation, audit logging, network restrictions and a documented retention policy. The current implementation should not be represented as a complete enterprise security control set.

13. CONCLUSION AND FUTURE ENHANCEMENTS

13.1 Conclusion

The Universal Log Processing Framework with AI Analysis demonstrates an integrated approach to collecting, normalising, streaming, classifying, indexing and investigating application logs. It addresses the practical difficulty of working with heterogeneous records by introducing a canonical representation and a sequence of independent processing stages. Kafka provides asynchronous transport, Elasticsearch supports structured and vector search, and the RAG pipeline uses historical log evidence to support AI-generated diagnosis.

The Streamlit dashboard brings the principal capabilities together through six pages. The implementation includes validation and masking, checkpoint-based collection, classification and priority assignment, separate dead-letter topics, searchable records, semantic retrieval and diagnosis review. The supplied development

evidence confirms that the pages render and that retained log, diagnosis and failure records are available for inspection.

The project is best understood as an investigation-support framework. Its AI output is advisory, the dashboard does not resolve incidents automatically, and further work is required before the system could be treated as a production-grade observability platform.

13.2 Future Enhancements

- Strengthen access and service security with dashboard roles, TLS and authenticated Kafka and Elasticsearch connections.
- Add configurable DLQ replay and retry workflows, with idempotency safeguards and audit trails.
- Introduce incident lifecycle management for acknowledgement, assignment, status changes and resolution notes.
- Correlate logs with metrics and distributed traces to provide a wider view of system behaviour.
- Add schema versioning and compatibility tests for changes to canonical log records.
- Run repeatable throughput, latency, load and failure-recovery tests under defined conditions.
- Provide configurable alerts for critical patterns and DLQ growth, including message-count and storage thresholds.
- Evaluate retrieval quality and diagnosis accuracy against a labelled historical incident dataset.
- Package services with deployment profiles, health checks and centralised configuration for multi-node environments.
- Set DLQ retention and cleanup policies, with storage quotas and resource limits to prevent unbounded accumulation.

REFERENCES

[1] Apache Software Foundation. Apache Kafka Documentation. https://kafka.apache.org/documentation/

[2] Elastic. Elasticsearch Guide, version 8.17. https://www.elastic.co/guide/en/elasticsearch/reference/8.17/index.html

[3] Streamlit, Inc. Streamlit Documentation. https://docs.streamlit.io/

[4] UKP Lab / Hugging Face. Sentence Transformers Documentation. https://www.sbert.net/

[5] Groq, Inc. Groq API Documentation. https://console.groq.com/docs

[6] Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/

[7] Project source code: Universal Log Processing Framework with AI Analysis, supplied project repository snapshot and implementation files (accessed November 2026).

[8] Project verification material: local command output, dashboard screenshots and development test observations supplied for this report (November 2026).

[9] Python Software Foundation. Python Standard Library — XML Processing. https://docs.python.org/3/library/xml.html

[10] Python Software Foundation. Python Standard Library — CSV File Reading and Writing. https://docs.python.org/3/library/csv.html

[11] Gerhards, R. The Syslog Protocol (RFC 5424). Internet Engineering Task Force (IETF). https://www.rfc-editor.org/rfc/rfc5424

APPENDIX A — KAFKA TOPIC INVENTORY

Topic	Purpose
normalized-logs	Canonical events awaiting processing.
processed-logs	Events after classification and priority assignment.
dead-letter-logs	Events rejected by log processing.
ai-diagnosis	AI diagnosis results.
ai-diagnosis-dlq	AI diagnosis processing failures.
__consumer_offsets	Kafka internal consumer offset metadata.

APPENDIX B — PRIORITY INTERPRETATION

The implemented priority mapping is a project-specific rule set. P1 is assigned to CRITICAL, P2 to ERROR, P3 to WARNING, P4 to INFO and P5 to DEBUG. A priority value is a classification output; it does not independently measure business impact, affected-user count or incident urgency.

APPENDIX C — OPERATIONAL START-UP CHECKLIST

1. Activate the project Python virtual environment.
2. Start the Kafka Docker container and verify that the expected topics are present.
3. Start the Elasticsearch Docker container and confirm that its HTTP endpoint responds.
4. Check that the environment configuration is available without displaying secret values.
5. Start the collector, processing, indexing and AI diagnosis services required for the run.
6. Launch the Streamlit application from the project root using the documented command.
7. Open the dashboard pages and confirm that their required data sources are available.

APPENDIX D — EVIDENCE AND REPRODUCIBILITY NOTE

The screenshots and diagrams in this report were taken from the project materials and the running application. Implementation details were checked against the available source files, configuration and local test observations. Results are reported only where supporting evidence was available. The report does not reproduce API keys, passwords or the contents of the project .env file.